AI SMPS 2023 Week 3 Algorithms (Ver. 2)

Prepared by S. Baskaran

Node Order

MoveGen determines the order in which nodes are generated, and algorithms determine the order in which nodes are inspected. Some algorithms follow MoveGen order and some reorder the nodes.

For each algorithm discussed in this course, study the order in which nodes are added-to and removed-from OPEN/CLOSED lists and other data structures.

It is important to understand how list structures are constructed, accessed and printed. Study the 'cons' and 'append' operators and pay attention to the order in which items are cons-ed (added) and appended to lists.

WARNING: while answering auto-graded short-answer type questions, it is important to maintain the algorithm specific node order, otherwise, the auto-grader will mark the answer as wrong and there are no partial marks.

WARNING: often, participants understand the concepts correctly and they may follow a different variation of an algorithm or follow a different implementation of cons and append operators and arrive at a practically viable solution but with a different node order, in this case the auto-grader will mark the answer as wrong.

Tie Breaking a Pool of Best Nodes

MoveGen determines the order in which nodes are generated, and algorithms determine the order in which nodes are inspected. Some algorithms follow MoveGen order and some reorder the nodes.

Cost based (deterministic or heuristic) algorithms select the best node in each iteration. Often there will be a pool of best nodes with the same cost and we do not know which nodes lead to a solution.

In a real world scenario, an additional criteria (external to the algorithm) may be provided to filter the pool of best nodes, or else a random node may be selected from the pool. Each such selection may potentially lead to a different solution or no solution at all.

A pool of best nodes make it difficult to setup all possible answers for an auto-graded question, so we use node label to break ties.

> When multiple nodes have the same (best) cost, sort those nodes in alphabetical order of node label and select nodes from the head of the sorted list.

This tie-breaker helps us to focus on the algorithm and not get beat up on the finer details of a (made up) toy problem.

On the other hand, outside the scope of auto-graded assignments and exams, we urge you to investigate all the problems/solutions/algorithms and find ways to improve it. Wish you the best.

Algorithms

This is a continuation of algorithms published in Week 2 Notes.

```
BEST-FIRST-SEARCH(S)
 1  OPEN ← (S, null, h(S)) : []
 2  CLOSED ← empty list
 3  while OPEN is not empty
 4      nodePair ← head OPEN
 5      (N, _, _) ← nodePair
 6      if GOALTEST(N) = TRUE
 7          return RECONSTRUCTPATH(nodePair, CLOSED)
 8      else CLOSED ← nodePair : CLOSED
 9          neighbours ← MOVEGEN(N)
10          newNodes ← REMOVESEEN(neighbours, OPEN, CLOSED)
11          newPairs ← MAKEPAIRS(newNodes, N)
12          OPEN ← sort_h ( newPairs ++ tail OPEN )
13  return empty list
```

```
HILL-CLIMBING(S)
1  N ← S
2  do  bestEver ← N
3      N ← head sort_h MOVEGEN(bestEver)
4  while h(N) is better than h(bestEver)
5  return bestEver
```

```
VARIABLE-NEIGHBOURHOOD-DESCENT(S)
1  MoveGenList ← MOVEGEN_1 : MOVEGEN_2 : ··· : MOVEGEN_n : []
2  bestNode ← S
3  while MoveGenList is not empty
4      bestNode ← HILL-CLIMBING(bestNode, head MoveGenList)
5      MoveGenList ← tail MoveGenList
6  return bestNode
```

```
BEST-NEIGHBOUR-SEARCH(S)
1  N ← S
2  bestEver ← S
3  until some termination condition
4      N ← best MOVE-GEN(N)
5      if N is better than bestEver
6          bestEver ← N
7  return bestEver
```

```
ITERATED-HILL-CLIMBING(N)
1  bestNode ← random candidate solution
2  repeat N times
3      currentBest ← HILL-CLIMBING(new random candidate solution)
4      if h(currentBest) is better than h(bestNode)
5          bestNode ← currentBest
6  return bestNode
```

Beam Search

We will use the following version of Beam Search in assignments and exams because it is assignment friendly and prevents infinite loops.

Here, $\textbf{sort}_\textbf{h}$ sorts from best to worst **h**-values.

```
BEAM-SEARCH(S, w)
 1  OPEN ← S : []
 2  N ← S
 3  do  bestEver ← N
 4      if OPEN contains goal node
 5          return that goal node
 6      else neighbours ← MOVE-GEN(OPEN)
 7          OPEN ← take w (sort_h neighbours)
 8          N ← head OPEN      ▷ best in new layer
 9  while h(N) is better than h(bestEver)
10  return bestEver
```

```
MOVE-GEN(OPEN)
1  neighbours ← []
2  for each X in OPEN
3      neighbours ← neighbours ++ MOVE-GEN(X)
4  return neighbours     ▷ the list preserves duplicates
```

(**take** n LIST) returns at most n values from the beginning of LIST.

[o, u, t] = **take** 3 [o, u, t, r, u, n]
[a, t] = **take** 3 [a, t]
[a] = **take** 3 [a]
[] = **take** 3 []

TSP Algorithms

NEAREST-NEIGHBOUR-HEURISTIC

1 Compute the cost of all the edges
2 Start at some city
3 **repeat**
4 Move to the nearest neighbour which does not:
close the loop prematurely
or form branches/forks

GREEDY-HEURISTIC

1 Compute the cost of all the edges
2 Sort the edges by cost
3 **repeat**
4 Add the shortest available edge which does not:
close the loop prematurely
or form branches/forks

SAVINGS-HEURISTIC

```
1  n ← select a base city from N cities          ▷ base city
2  construct (N − 1) sub-tours, each of length 2 anchored at n

3  savingsList ← empty list
4  for each non base city a
5      for each non base city b, if b > a        ▷ ignores inverse edge
6          savings ← cost(n, a) + cost(n, b) − cost(a, b)
7          savingsList ← (a, b, savings) : savingsList

8  sort savingsList in descending order of savings

9  for each tuple (a, b, _) in savingsList
10     if edges (n, a) and (n, b) are in two different tours
           ▷ merge the two tours
11         remove edges (n, a) and (n, b)
12         insert the edge (a, b)
13 return the final tour
```